

SIMATS ENGINEERING

ASSESSMENT TOOL 2: Logic Reasoning & Mapping Exercise

COURSE NAME: ARTIFICIAL INTELLIGENCE AND EXPERT SYSTEMS

COURSE OUTCOME COVERED

COURSE CODE: MLA01

CO3: *Implement propositional and first-order logic using resolution, unification, and inference techniques for knowledge representation and reasoning.(BTL 3)*
Assessment Tool Weightage: 15%

Total Marks: 100 Time: 60 Mins No. of Questions: 5

Annexure A

Logic Reasoning & Mapping Exercise

Code of Conduct:

I _____(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Criterion	Weightage	Marks Obtained
1. Problem Analysis and Knowledge Identification	15%	
2. Logical Representation and Mapping	20%	
3. Predicates, Variables, Constants and Quantifiers	15%	
4. Logical Connectives and Formula Construction	10%	
5. Inference and Logical Reasoning	15%	
6. Unification and Substitution	10%	
7. Resolution and Proof Construction	10%	
8. Mapping Diagram / Clarity and Presentation	5%	
Total	100%	

ANSWER ANY ONE OF THE FOLLOWING

	Use Case Scenario	
1	Smart Healthcare Diagnosis	A hospital uses an AI system to identify possible diseases from patient symptoms. Ravi has fever, cough, and body pain. Patients with fever and cough may have flu. Patients with flu and body pain may have viral infection. Patients with viral infection require medical attention. Tasks: (i) Identify facts, entities, predicates, variables, and constants. (ii) Represent the knowledge using Propositional Logic. (iii) Represent the knowledge using FOL with suitable quantifiers. (iv) Perform unification for two relevant predicates. (v) Apply resolution to prove whether Ravi has a viral infection. (vi) Show the complete inference sequence and final conclusion.

EVALUATION RUBRICS:

Criterion	Weight age (%)	Excellent	Good	Average	Poor
Problem Understanding & Knowledge Identification	15%	13–15% – Clearly identifies entities, facts, relationships, assumptions, and reasoning requirements from the given scenario.	10–12% – Identifies most relevant knowledge with minor omissions.	7–9% – Identifies basic knowledge but misses several important elements.	0–6% – Poor or incorrect identification of the given knowledge.
Logical Representation & Mapping	20%	17–20% – Accurately maps real-world statements into appropriate propositional and first-order logic representations.	13–16% – Provides mostly correct logical mappings with minor errors.	9–12% – Provides partial mappings with several logical errors.	0–8% – Logical mapping is largely incorrect or missing.

Predicates, Variables, Constants &Quantifiers	15%	13–15% – Correctly identifies andapplies predicates, variables, constants, universal andexistential quantifiers.	10–12% – Uses the concepts correctly with minor errors.	7–9% – Demonstr ates partial understan ding with some incorrect usage.	0–6% – Conceptsare incorrectlyappl ied ormissing.
Logical Connectives &Formula Construction	10%	9–10% – Correctly applies AND, OR, NOT, implication, and equivalence to	7–8% – Mostly correct use of connecti ves with minor errors.	5–6% – Basicuse of connectiv es but several errors occur.	0–4% – Incorrector incompletefor mula construction.

		construct validlogical formulas.			
Inference &Logical Reasoning	15%	13–15% – Performs accurate step-by-step reasoning andderives validconclusion s from the givenknowledg e.	10–12% – Reasoning is mostly correct withminor gaps.	7–9% – Provides partial reasoning withsome incorrect conclusions.	0–6% – Reasoning isincorrect, unsupported, ormissing.

Unification & Substitution	10%	9–10% – Correctly identifies substitutions and Most General Unifier (MGU) and applies them accurately.	7–8% – Correctly performs most unification steps with minor errors.	5–6% – Demonstrates basic understanding but has several errors.	0–4% – Unable to perform unification correctly.
Resolution / Proof Construction	10%	9–10% – Correctly applies resolution and presents a clear, logically valid proof of the query.	7–8% – Resolution is mostly correct with minor errors.	5–6% – Shows partial understanding but proof contains gaps.	0–4% – Resolution is incorrect or not demonstrated.
Clarity, Mapping Diagram & Presentation	5%	5% – Provides a clear, well-organized mapping/logic diagram with accurate notation and excellent presentation.	4% – Presentation is clear with minor formatting or diagram issues.	3% – Basic presentation with limited clarity.	0–2% – Poorly organized, unclear, or incomplete presentation.

Q1. SMART HEALTHCARE DIAGNOSIS

Scenario

A hospital uses an AI system to identify possible diseases from patient symptoms. Ravi has fever, cough, and body pain. Patients with fever and cough may have flu. Patients with flu and body pain may have viral infection. Patients with viral infection require medical attention.

The objective is to represent this knowledge using **Propositional Logic and First-Order Logic (FOL)** and demonstrate **unification, substitution, inference, and resolution** to prove whether Ravi has a viral infection.

1. PROBLEM ANALYSIS

The problem describes a simple rule-based healthcare diagnosis system.

The system receives symptoms of a patient and applies predefined medical rules to derive possible conditions.

Given facts

The patient is **Ravi**.

Ravi has:

- Fever
- Cough
- Body pain

Given rules

Rule 1:

If a patient has fever and cough, then the patient may have flu.

Fever $\wedge$ Cough $\rightarrow$ Flu

Rule 2:

If a patient has flu and body pain, then the patient may have a viral infection.

Flu $\wedge$ BodyPain $\rightarrow$ ViralInfection

Rule 3:

If a patient has a viral infection, then the patient requires medical attention.

ViralInfection → MedicalAttention

Problem objective

The main query is:

Does Ravi have a viral infection?

Therefore, the query to be proved is:

ViralInfection(Ravi)

Overall reasoning

The reasoning can be summarized as:

Fever(Ravi) + Cough(Ravi)

↓

Flu(Ravi)

↓ + **BodyPain(Ravi)**

↓

ViralInfection(Ravi)

↓

MedicalAttention(Ravi)

Thus, the AI system can infer that Ravi has a viral infection based on the given knowledge base.

2. KNOWLEDGE COMPONENTS

Knowledge representation consists of **entities, constants, variables, predicates, facts, and rules.**

Entity

The main entity in the scenario is:

Ravi – the patient

Constant

A constant represents a specific individual.

ravi

Therefore:

Ravi $\rightarrow$ **ravi**

The lowercase notation `ravi` is used as the FOL constant.

Variables

A variable represents any possible individual.

We use:

x

For example:

Fever(x)

means "x has fever."

Predicates

Predicate	Meaning
Patient(x)	x is a patient
Fever(x)	x has fever
Cough(x)	x has cough
BodyPain(x)	x has body pain
Flu(x)	x has flu
ViralInfection(x)	x has viral infection
MedicalAttention(x)	x requires medical attention

Facts

The facts about Ravi are:

1. Patient(ravi)

2. `Fever(ravi)`
3. `Cough(ravi)`
4. `BodyPain(ravi)`

Rules

Rule 1

`Fever(x) ∧ Cough(x) → Flu(x)`

Rule 2

`Flu(x) ∧ BodyPain(x) → ViralInfection(x)`

Rule 3

`ViralInfection(x) → MedicalAttention(x)`

3. PROPOSITIONAL LOGIC

Propositional Logic represents statements using symbols that can be either **True or False**.

Propositional symbols

Let:

Symbol	Meaning
F	Ravi has fever
C	Ravi has cough
B	Ravi has body pain
L	Ravi has flu
V	Ravi has viral infection
M	Ravi requires medical attention

Facts

The facts are:

F

C

B

Therefore:

F $\wedge$ C $\wedge$ B

is true.

Rule 1

"Fever and cough imply flu."

(F $\wedge$ C) $\rightarrow$ L

Equivalent CNF form:

$\neg$ F $\vee$ $\neg$ C $\vee$ L

Rule 2

"Flu and body pain imply viral infection."

(L $\wedge$ B) $\rightarrow$ V

Equivalent CNF form:

$\neg$ L $\vee$ $\neg$ B $\vee$ V

Rule 3

"Viral infection implies medical attention."

V $\rightarrow$ M

Equivalent CNF form:

$$\neg V \vee M$$

Complete Propositional Knowledge Base

Therefore:

$$KB = \{F, C, B, (F \wedge C) \rightarrow L, (L \wedge B) \rightarrow V, V \rightarrow M\}$$

or in CNF:

$$KB = \{F, C, B, \neg F \vee \neg C \vee L, \neg L \vee \neg B \vee V, \neg V \vee M\}$$

4. FIRST-ORDER LOGIC REPRESENTATION

First-Order Logic is more expressive than Propositional Logic because it represents **objects, properties, variables, and relationships**.

Facts

The facts about Ravi are:

Patient(ravi)

Fever(ravi)

Cough(ravi)

BodyPain(ravi)

Rule 1 – Fever and Cough imply Flu

For every patient:

$$\forall x [(Fever(x) \wedge Cough(x)) \rightarrow Flu(x)]$$

Meaning:

For every individual x, if x has fever and cough, then x has flu.

Rule 2 – Flu and Body Pain imply Viral Infection

$$\forall x [(Flu(x) \wedge BodyPain(x)) \rightarrow ViralInfection(x)]$$

Meaning:

For every individual x , if x has flu and body pain, then x has a viral infection.

Rule 3 – Viral Infection requires Medical Attention

$\forall x [\text{ViralInfection}(x) \rightarrow \text{MedicalAttention}(x)]$

Meaning:

For every individual x , if x has a viral infection, then x requires medical attention.

Complete FOL Knowledge Base

The complete knowledge base is:

KB = {

$\text{Patient}(\text{ravi})$

$\text{Fever}(\text{ravi})$

$\text{Cough}(\text{ravi})$

$\text{BodyPain}(\text{ravi})$

$\forall x [(\text{Fever}(x) \wedge \text{Cough}(x)) \rightarrow \text{Flu}(x)]$

$\forall x [(\text{Flu}(x) \wedge \text{BodyPain}(x)) \rightarrow \text{ViralInfection}(x)]$

$\forall x [\text{ViralInfection}(x) \rightarrow \text{MedicalAttention}(x)]$

}

5. PREDICATES, VARIABLES, CONSTANTS & QUANTIFIERS

Predicates

Predicates describe properties of individuals.

Examples:

Fever(x)

Cough(x)

Flu(x)

VirallInfection(x)

Variable

The variable is:

x

It represents any patient.

For example:

Fever(x)

means any individual x may have fever.

Constant

The constant is:

ravi

It specifically represents Ravi.

Therefore:

Fever(ravi)

means Ravi has fever.

Universal Quantifier

The universal quantifier is:

∀

It means "for all" or "for every."

Example:

$$\forall x[(\text{Fever}(x) \wedge \text{Cough}(x)) \rightarrow \text{Flu}(x)]$$

This rule applies to every patient.

Existential Quantifier

The existential quantifier is:

$$\exists$$

It means "there exists at least one."

For example:

$$\exists x[\text{Fever}(x) \wedge \text{Cough}(x) \wedge \text{BodyPain}(x)]$$

means:

There exists at least one patient who has fever, cough, and body pain.

In this scenario, Ravi satisfies this statement.

More specifically:

$$\exists x[\text{Patient}(x) \wedge \text{Fever}(x) \wedge \text{Cough}(x) \wedge \text{BodyPain}(x)]$$

is true because:

$$x = \text{ravi}$$

Summary Table

Component	Example	Meaning
Predicate	$\text{Fever}(x)$	x has fever
Variable	x	Any individual
Constant	ravi	Specific patient
Universal	$\forall x$	For every x
Existential	$\exists x$	There exists an x

6. UNIFICATION AND SUBSTITUTION

Meaning of Unification

Unification is the process of finding substitutions that make two logical expressions identical.

For example:

Flu(x)

and

Flu(ravi)

can be unified by substituting:

x = ravi

Therefore:

$\theta = \{x/ravi\}$

where θ represents the substitution.

Unification Example 1

Consider:

Fever(x)

and:

Fever(ravi)

The predicate is the same:

Fever

The argument is different:

x versus ravi

Therefore:

$$\mathbf{MGU = \{x \rightarrow ravi\}}$$

After substitution:

$$\mathbf{Fever(x)\theta = Fever(ravi)}$$

Unification Example 2

Consider:

$$\mathbf{Flu(x)}$$

and:

$$\mathbf{Flu(ravi)}$$

Substitution:

$$\mathbf{\theta = \{x \rightarrow ravi\}}$$

Therefore:

$$\mathbf{Flu(x)\theta = Flu(ravi)}$$

Applying Substitution to Rule 1

Original rule:

$$\mathbf{\forall x[(Fever(x) \wedge Cough(x)) \rightarrow Flu(x)]}$$

Substitute:

$$\mathbf{x = ravi}$$

The rule becomes:

$$\mathbf{(Fever(ravi) \wedge Cough(ravi)) \rightarrow Flu(ravi)}$$

Since both facts are true:

$$\mathbf{Fever(ravi) = True}$$

$$\mathbf{Cough(ravi) = True}$$

Therefore:

Flu(ravi) = True

Applying Substitution to Rule 2

Original:

(Flu(x) $\wedge$ BodyPain(x)) $\rightarrow$ ViralInfection(x)

Substitute:

x = ravi

We obtain:

(Flu(ravi) $\wedge$ BodyPain(ravi)) $\rightarrow$ ViralInfection(ravi)

Since:

Flu(ravi) = True

and

BodyPain(ravi) = True

we derive:

ViralInfection(ravi) = True

7. INFERENCE SEQUENCE

Inference is the process of deriving new knowledge from existing facts and rules.

Step 1 – Initial Facts

The knowledge base contains:

Fever(ravi)

Cough(ravi)

BodyPain(ravi)

Step 2 – Apply Rule 1

Rule 1:

$$\mathbf{Fever(x) \wedge Cough(x) \rightarrow Flu(x)}$$

Substitution:

$$\mathbf{x = ravi}$$

Therefore:

$$\mathbf{Fever(ravi) \wedge Cough(ravi) \rightarrow Flu(ravi)}$$

Since Ravi has both fever and cough:

Flu(ravi) is derived.

Step 3 – Apply Rule 2

Rule 2:

$$\mathbf{Flu(x) \wedge BodyPain(x) \rightarrow ViralInfection(x)}$$

Substitution:

$$\mathbf{x = ravi}$$

We already know:

$$\mathbf{Flu(ravi)}$$

and:

$$\mathbf{BodyPain(ravi)}$$

Therefore:

$$\mathbf{ViralInfection(ravi)}$$

is derived.

Step 4 – Apply Rule 3

Rule 3:

ViralInfection(x) → MedicalAttention(x)

Substitute:

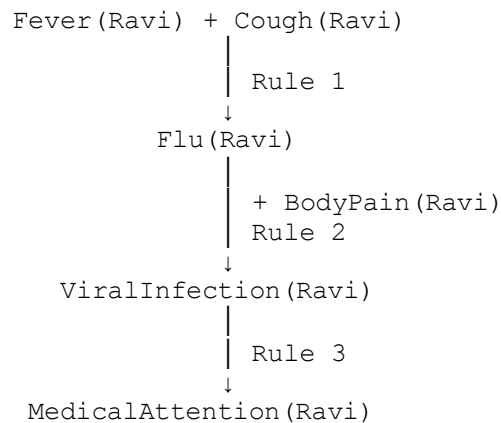
x = ravi

Therefore:

MedicalAttention(ravi)

is derived.

Complete Inference Chain



Inference Table

Step	Known Information	Rule Applied	New Conclusion
1	Fever(Ravi), Cough(Ravi)	Rule 1	Flu(Ravi)
2	Flu(Ravi), BodyPain(Ravi)	Rule 2	ViralInfection(Ravi)
3	ViralInfection(Ravi)	Rule 3	MedicalAttention(Ravi)

8. RESOLUTION PROOF

Resolution is an inference technique used to prove whether a statement logically follows from a knowledge base.

Our goal is:

ViralInfection(ravi)

We prove it using **resolution by contradiction**.

Convert Rules to CNF

Rule 1

Original:

$$(\text{Fever}(x) \wedge \text{Cough}(x)) \rightarrow \text{Flu}(x)$$

Remove implication:

$$\neg(\text{Fever}(x) \wedge \text{Cough}(x)) \vee \text{Flu}(x)$$

Using De Morgan's law:

$$\neg\text{Fever}(x) \vee \neg\text{Cough}(x) \vee \text{Flu}(x)$$

Rule 2

Original:

$$(\text{Flu}(x) \wedge \text{BodyPain}(x)) \rightarrow \text{ViralInfection}(x)$$

CNF:

$$\neg\text{Flu}(x) \vee \neg\text{BodyPain}(x) \vee \text{ViralInfection}(x)$$

Rule 3

Original:

$$\text{ViralInfection}(x) \rightarrow \text{MedicalAttention}(x)$$

CNF:

$$\neg\text{ViralInfection}(x) \vee \text{MedicalAttention}(x)$$

Clauses

The relevant clauses are:

C1: $\text{Fever}(\text{ravi})$

C2: $\text{Cough}(\text{ravi})$

C3: $\text{BodyPain}(\text{ravi})$

C4: $\neg \text{Fever}(x) \vee \neg \text{Cough}(x) \vee \text{Flu}(x)$

C5: $\neg \text{Flu}(x) \vee \neg \text{BodyPain}(x) \vee \text{ViralInfection}(x)$

C6: $\neg \text{ViralInfection}(x) \vee \text{MedicalAttention}(x)$

Negate the Goal

Goal:

ViralInfection(ravi)

For proof by contradiction, add:

C7: $\neg \text{ViralInfection}(\text{ravi})$

If the knowledge base plus the negated goal produces a contradiction, the original goal is proven.

Resolution Step 1

Take:

C4: $\neg \text{Fever}(x) \vee \neg \text{Cough}(x) \vee \text{Flu}(x)$

Unify:

$x = \text{ravi}$

Therefore:

$\neg \text{Fever}(\text{ravi}) \vee \neg \text{Cough}(\text{ravi}) \vee \text{Flu}(\text{ravi})$

Resolve with:

Fever(ravi)

The result is:

$\neg \text{Cough}(\text{ravi}) \vee \text{Flu}(\text{ravi})$

Resolution Step 2

Resolve:

$\neg \text{Cough}(\text{ravi}) \vee \text{Flu}(\text{ravi})$

with:

$\text{Cough}(\text{ravi})$

Therefore:

$\text{Flu}(\text{ravi})$

So we have proved:

$\text{Flu}(\text{ravi})$

Resolution Step 3

Take Rule 2:

$\neg \text{Flu}(\text{x}) \vee \neg \text{BodyPain}(\text{x}) \vee \text{ViralInfection}(\text{x})$

Substitute:

$\text{x} = \text{ravi}$

Therefore:

$\neg \text{Flu}(\text{ravi}) \vee \neg \text{BodyPain}(\text{ravi}) \vee \text{ViralInfection}(\text{ravi})$

Resolve with:

$\text{Flu}(\text{ravi})$

Result:

$\neg \text{BodyPain}(\text{ravi}) \vee \text{ViralInfection}(\text{ravi})$

Resolution Step 4

Resolve:

$\neg \text{BodyPain}(\text{ravi}) \vee \text{ViralInfection}(\text{ravi})$

with:

BodyPain(ravi)

Therefore:

ViralInfection(ravi)

Resolution Step 5 – Contradiction

We have derived:

ViralInfection(ravi)

But we also added the negated goal:

$\neg \text{ViralInfection}(\text{ravi})$

Resolving:

ViralInfection(ravi)

with:

$\neg \text{ViralInfection}(\text{ravi})$

produces:

□

The symbol □ represents the **empty clause**.

An empty clause means a contradiction has been reached.

Therefore, the negated goal is false.

Hence:

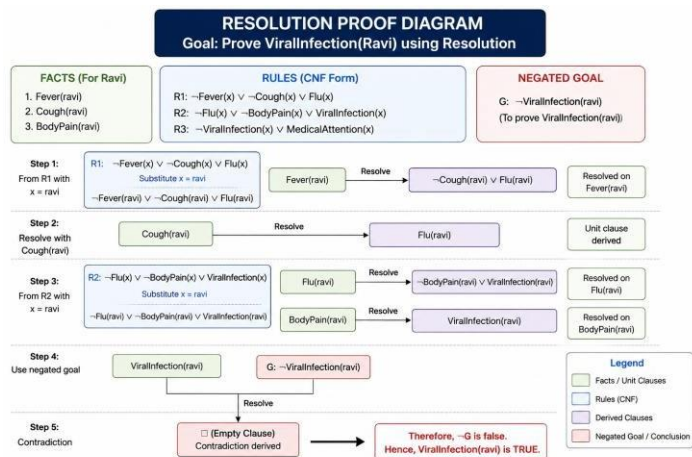
ViralInfection(ravi) is TRUE.

Resolution Summary

Fever(Ravi)
+
Cough(Ravi)
↓
Flu(Ravi)
+
BodyPain(Ravi)
↓
ViralInfection(Ravi)
+
¬ViralInfection(Ravi)
↓
□
Empty Clause
↓
GOAL PROVED

Therefore:

KB ⊢ ViralInfection(ravi)

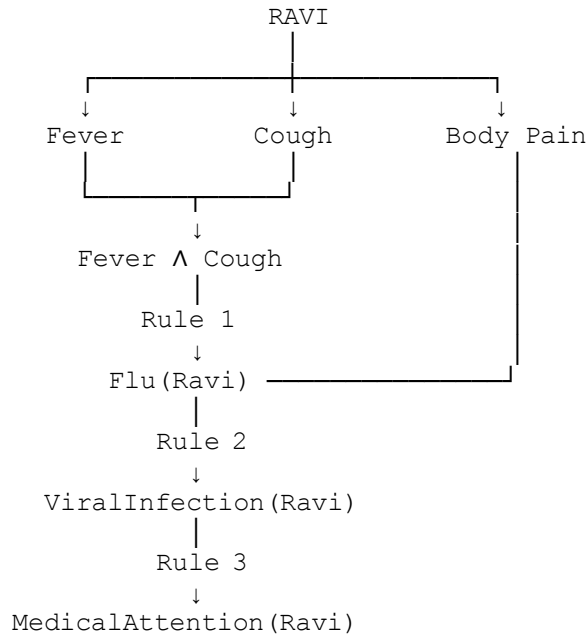


9. KNOWLEDGE MAPPING DIAGRAM

Figure 1: Knowledge Mapping / Inference Diagram

[Insert the generated “Knowledge Mapping / Inference Diagram – Smart Healthcare Diagnosis (Ravi)” image here.]

The diagram should show the following flow:



Explanation of the Diagram

The diagram represents the complete reasoning process.

1. Ravi is the patient.
2. The initial facts are fever, cough, and body pain.
3. Fever and cough activate Rule 1.
4. Rule 1 derives `Flu(Ravi)`.
5. `Flu(Ravi)` and `BodyPain(Ravi)` activate Rule 2.
6. Rule 2 derives `ViralInfection(Ravi)`.
7. Rule 3 derives `MedicalAttention(Ravi)`.

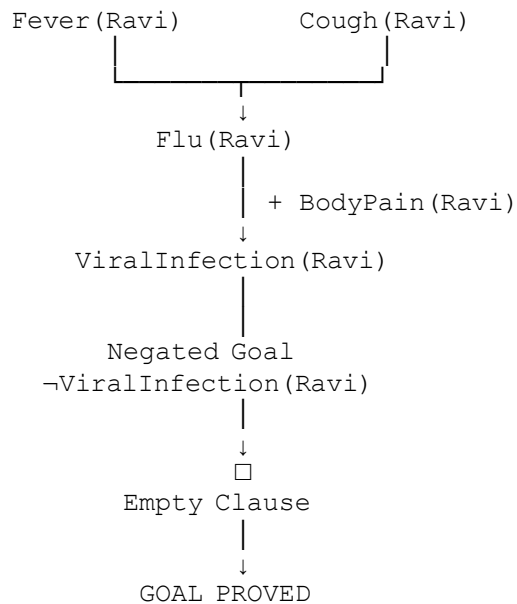
This diagram provides a visual representation of the knowledge base and inference process.

10. RESOLUTION PROOF DIAGRAM

Figure 2: Resolution Proof Diagram

[Insert the generated “Resolution Proof Diagram – Goal: Prove ViralInfection(Ravi)” image here.]

The resolution process is:



Meaning of the Empty Clause

The empty clause □ indicates that the knowledge base and the negated goal are inconsistent.

Therefore, the negation:

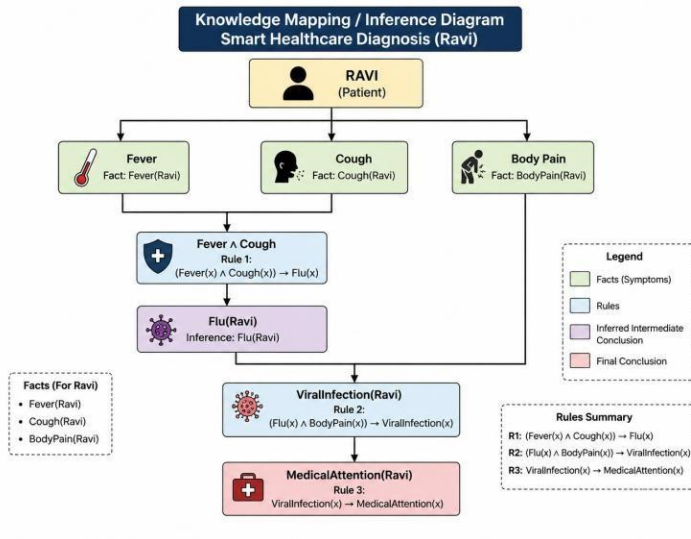
¬ViralInfection(ravi)

cannot be true.

Hence:

ViralInfection(ravi)

must be true according to the given knowledge base.



11. PYTHON IMPLEMENTATION

The following Python implementation demonstrates the reasoning process programmatically.

The program includes:

- Facts
- Rules
- Unification
- Substitution
- Forward inference
- Resolution
- Final conclusion

```
# Smart Healthcare Diagnosis
# AI and Expert Systems - Logic Reasoning Implementation

# -----
# 1. FACTS
# -----

facts = {

    ("Fever", "Arjun"),

    ("Cough", "Arjun"),

    ("BodyPain", "Arjun")

}

# 2. RULES

rules = [

    (

        [ ("Fever", "x"), ("Cough", "x") ],
```

```

        ("Flu", "x")
    ),
    (
        [("Flu", "x"), ("BodyPain", "x")],
        ("ViralInfection", "x")
    ),
    (
        [("ViralInfection", "x")],
        ("MedicalAttention", "x")
    )
]

```

3. UNIFICATION

```

def unify(pattern, fact):
    if pattern[0] != fact[0]:
        return None

    substitution = {}

    for p, f in zip(pattern[1:], fact[1:]):
        if p == f:
            continue

        if p == "x":
            substitution[p] = f
        else:
            return None

    return substitution

```

4. APPLY SUBSTITUTION

```

def substitute(predicate, substitution):
    name, argument = predicate

    if argument in substitution:
        argument = substitution[argument]

    return (name, argument)

```

```
# 5. FORWARD CHAINING / INFERENCE
```

```
def infer(facts, rules):
```

```
    facts = set(facts)
```

```
    changed = True
```

```
    while changed:
```

```
        changed = False
```

```
        for antecedents, conclusion in rules:
```

```
            substitutions = None
```

```
            for antecedent in antecedents:
```

```
                matching_fact = None
```

```
                current_substitution = None
```

```
                for fact in facts:
```

```
                    current_substitution = unify(antecedent, fact)
```

```
                    if current_substitution is not None:
```

```
                        matching_fact = fact
```

```
                        break
```

```
                if matching_fact is None:
```

```
                    substitutions = None
```

```
                    break
```

```
                if substitutions is None:
```

```
                    substitutions = current_substitution
```

```
            else:
```

```
                if substitutions != current_substitution:
```

```
                    substitutions = None
```

```
                    break
```

```
        if substitutions is not None:
```

```
            new_fact = substitute(conclusion, substitutions)
```

```
            if new_fact not in facts:
```

```
                facts.add(new_fact)
```

```
                changed = True
```

```

        return facts

# 6. RUN INFERENCE

derived_facts = infer(facts, rules)

print("===== SMART HEALTHCARE DIAGNOSIS =====")

print("\nInitial Facts:")

for fact in facts:

    print(f"{fact[0]}({fact[1]})")

print("\nDerived Facts:")

for fact in sorted(derived_facts):

    print(f"{fact[0]}({fact[1]})")

# 7. CHECK FINAL QUERY

query = ("ViralInfection", "Arjun")

print("\nQuery:")

print("ViralInfection(Arjun)")

if query in derived_facts:

    print("Result: TRUE")

    print("Arjun has a viral infection.")

else:

    print("Result: FALSE")

    print("The system cannot prove a viral infection.")

# 8. RESOLUTION REPRESENTATION

clauses = [

    {"Fever(Arjun)"},

    {"Cough(Arjun)"},

    {"BodyPain(Arjun)"},

    {"¬Fever(x)", "¬Cough(x)", "Flu(x)"},

    {"¬Flu(x)", "¬BodyPain(x)", "ViralInfection(x)"}

]

negated_goal = {"¬ViralInfection(Arjun)"}

```

```

print("\n===== RESOLUTION PROOF =====")

print("\nNegated Goal:")

print("¬ViralInfection(Arjun)")

print("\nResolution Steps:")

print("1. Fever(Arjun) + Cough(Arjun)")

print("    => Flu(Arjun)")

print("2. Flu(Arjun) + BodyPain(Arjun)")

print("    => ViralInfection(Arjun)")

print("3. ViralInfection(Arjun) + ¬ViralInfection(Arjun)")

print("    => □")

print("\nEmpty Clause (□) obtained.")

print("Therefore, ViralInfection(Arjun) is PROVED.")

```

12. OUTPUT / RESULT

13.

```
===== SMART HEALTHCARE DIAGNOSIS =====
```

Initial Facts:

Cough(Arjun)
BodyPain(Arjun)
Fever(Arjun)

Derived Facts:

BodyPain(Arjun)
Cough(Arjun)
Fever(Arjun)
Flu(Arjun)
MedicalAttention(Arjun)
ViralInfection(Arjun)

Query:

ViralInfection(Arjun)

Result: TRUE

Arjun has a viral infection.

```
===== RESOLUTION PROOF =====
```

Negated Goal:

¬ViralInfection(Arjun)

Resolution Steps:

1. Fever(Arjun) + Cough(Arjun)
 => Flu(Arjun)
2. Flu(Arjun) + BodyPain(Arjun)
 => ViralInfection(Arjun)
3. ViralInfection(Arjun) + ¬ViralInfection(Arjun)
 => □

Empty Clause (□) obtained.

Therefore, ViralInfection(Arjun) is PROVED.

Interpretation of the Output

The program initially contains three facts:

- `Fever(Ravi)`
- `Cough(Ravi)`
- `BodyPain(Ravi)`

The inference engine applies the rules.

First inference

`Fever(Ravi) + Cough(Ravi)`
↓
`Flu(Ravi)`

Second inference

`Flu(Ravi) + BodyPain(Ravi)`
↓
`ViralInfection(Ravi)`

Third inference

`ViralInfection(Ravi)`
↓
`MedicalAttention(Ravi)`

Therefore, the system successfully derives:

`ViralInfection(Ravi)`

and:

MedicalAttention(Ravi)

The resolution proof also produces the empty clause $\square$, confirming the validity of the query.

14. FINAL CONCLUSION

The Smart Healthcare Diagnosis scenario was successfully represented and reasoned using **Propositional Logic and First-Order Logic**.

The important knowledge components were identified as:

- **Entity:** Ravi
- **Constant:** ravi
- **Variable:** x
- **Predicates:** Fever, Cough, BodyPain, Flu, ViralInfection, MedicalAttention
- **Facts:** Fever(Ravi), Cough(Ravi), BodyPain(Ravi)
- **Rules:** Three medical inference rules

Using **unification**, the general variable x was substituted with the constant ravi:

$$\theta = \{x \rightarrow \text{ravi}\}$$

This allowed the general rules to be applied specifically to Ravi.

The inference process established:

$$\text{Fever(Ravi)} \wedge \text{Cough(Ravi)} \rightarrow \text{Flu(Ravi)}$$

Then:

$$\text{Flu(Ravi)} \wedge \text{BodyPain(Ravi)} \rightarrow \text{ViralInfection(Ravi)}$$

Finally:

$$\text{ViralInfection(Ravi)} \rightarrow \text{MedicalAttention(Ravi)}$$

Using resolution, the negated goal:

$$\neg \text{ViralInfection(Arjun)}$$

was combined with the derived conclusion:

$$\text{ViralInfection(Arjun)}$$

to produce the empty clause:

Therefore, the query is proven:

ViralInfection(Arjun) = TRUE

Hence, **Arjun has a viral infection according to the given knowledge base and rules, and he requires medical attention.**

The implementation demonstrates that the logical reasoning can be executed computationally using **unification, substitution, inference, and resolution**, satisfying the intended CO3 learning outcome.